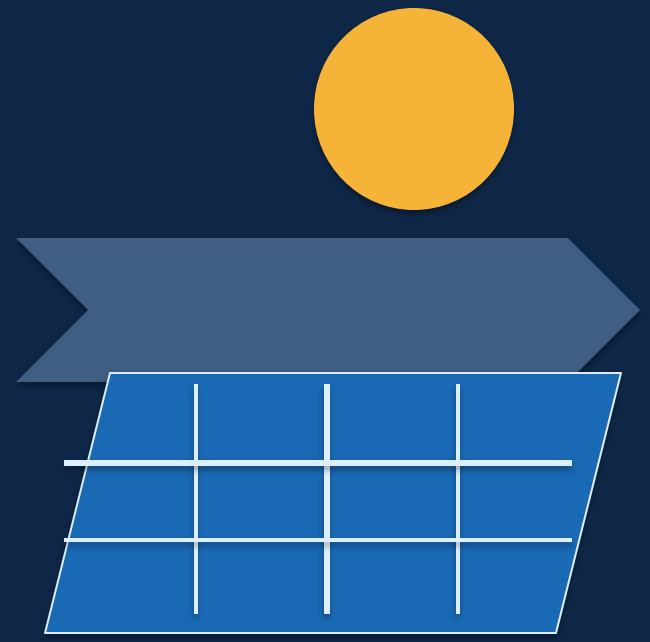
JAVA APPLICATION PROJECT

# ROOFTOP SOLAR ENERGY MONITOR



A console-based system to record solar generation, monitor energy use, and summarize performance.


- 2620030011 BALAJI.
- 2620030471 RAM CHARAN.

# Project Overview

What the system does and why it is useful

### Purpose

Record daily solar generation and household energy consumption.

### Users

Homeowners, students, and technicians who need a simple energy summary.

### Expected result

Clear readings, daily balance, alerts for unusual values, and saved reports.

## Core features

- Enter date, generated units (kWh), and consumed units (kWh).
- Calculate net energy and estimated generation value.
- Classify performance and flag invalid or low readings.
- View daily and monthly summaries.
- Search records by date and sort by energy generated.
- Save and load readings using a file.

# Understanding the Problem Statement

Problem, inputs, processing, and outputs

### Problem

Solar users may not have an easy way to compare generated energy with consumption. Manual calculations can be inconsistent and records can be lost.

### Proposed solution

Build a Java program that accepts readings, validates them, calculates metrics, organizes records, and saves summaries.

## Inputs

- Date / record ID
- Generated energy (kWh)
- Consumed energy (kWh)
- Tariff per unit (₹)

## Processing and outputs

- Net energy = generated − consumed
- Estimated value = generated × tariff
- Performance label and warnings
- Daily/monthly report and saved file

Assumption: educational console prototype; readings are entered manually, not collected from live hardware.

# System Design: Algorithm & Workflow

From user input to a useful energy report

### 1. Start

Display menu and initialize records.

### 2. Input

Read date, generation, consumption, and tariff.

### 3. Validate

Reject negative values and invalid choices.

### 4. Calculate

Compute net energy, value, and status.

### 5. Store

Keep records in collections and optionally save.

### 6. Report

Search, sort, summarize, and display results.

**Main loop: show menu → read choice → execute operation → repeat until the user selects Exit.**

# Module 1 / CO1 — Java Foundations

Primitive types, expressions, variables, and sequential console I/O

## Data representation

String date / record ID
double generatedKwh, consumedKwh, tariff
int menuChoice; boolean valid

## Expressions

netKwh = generatedKwh - consumedKwh
estimatedValue = generatedKwh * tariff
averageDaily = totalGenerated / numberOfDays
Use parentheses to make expression order clear.

## Sample console interaction

```
Enter generated energy (kWh): 12.5
Enter consumed energy (kWh): 8.0
Net energy: 4.5 kWh    Estimated value: ₹62.50
```

Concepts: variables, constants, operators, type casting, Scanner input, formatted output.

# CO1 — Java Foundations

```
import java.util.Scanner;

public class Module1_JavaFoundations {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);

        String date = "2026-10-14";
        String recordId = "R001";
        double tariff = 5.0;

        System.out.print("Enter generated energy (kWh): ");
        double generatedKwh = input.nextDouble();

        System.out.print("Enter consumed energy (kWh): ");
        double consumedKwh = input.nextDouble();

        double netKwh = generatedKwh - consumedKwh;
        double estimatedValue = generatedKwh * tariff;
        boolean valid = generatedKwh >= 0 && consumedKwh >= 0;

        if (valid) {
            System.out.println("\\nRecord ID: " + recordId);
            System.out.println("Date: " + date);
            System.out.printf("Net energy: %.2f kWh%n", netKwh);
            System.out.printf("Estimated value: Rs. %.2f%n", estimatedValue);
        } else {
            System.out.println("Invalid input. Energy values cannot be negative.");
        }
Input.close();
}
}
```

# Module 2 / CO2 — Selection & Iteration

Make decisions and repeat tasks safely

### if / else

If generation is below a threshold, display “Low generation”; otherwise display “Normal generation”.

### switch

Menu options: 1 Add reading, 2 View all, 3 Search, 4 Summary, 5 Save, 0 Exit.

### Loops

Use while or do-while to repeat the menu. Use for loops to process each reading.

## Example decision rules

- If generatedKwh < 0 or consumedKwh < 0 → invalid input.
- If generation ≥ consumption → non-negative balance; otherwise consumption exceeds generation.
- Use break in switch branches and repeat the menu until Exit.

# CO2 — Selection & Iteration (Part 1 of 2)

```
                                                                    import java.util.Scanner;

public class Module2_SelectionIteration {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        double generatedKwh = 12.5;
        double consumedKwh = 8.0;
        int choice;

        do {
            System.out.println("\\n--- Solar Monitor Menu ---");
            System.out.println("1. Add reading");
            System.out.println("2. View all");
            System.out.println("3. Search");
            System.out.println("4. Summary");
            System.out.println("5. Save");
            System.out.println("0. Exit");
            System.out.print("Choose an option: ");
            choice = input.nextInt();

            switch (choice) {
                case 1:
                    System.out.print("Enter generated kWh: ");
                    generatedKwh = input.nextDouble();
                    System.out.print("Enter consumed kWh: ");
                    consumedKwh = input.nextDouble();

                    if (generatedKwh < 0 || consumedKwh < 0) {
                        System.out.println("Invalid input: values cannot be negative.");
```

# CO2 — Selection & Iteration (Part 2 of 2)

```
                                                        } else if (generatedKwh >= consumedKwh) {
                        System.out.println("Normal generation: energy balance is non-negative.");
                    } else {
                        System.out.println("Low balance: consumption exceeds generation.");
                    }
                    break;
                case 2:
                    System.out.printf("Generated: %.2f kWh, Consumed: %.2f kWh%n",
                            generatedKwh, consumedKwh);
                    break;
                case 3:
                    System.out.println("Search example: enter a date in the full project.");
                    break;
                case 4:
                    System.out.printf("Net energy: %.2f kWh%n", generatedKwh - consumedKwh);
                    break;
                case 5:
                    System.out.println("Save option selected (file saving is in Module 5).");
                    break;
                case 0:
                    System.out.println("Exiting Solar Monitor.");
                    break;
                default:
                    System.out.println("Invalid option. Please try again.");
            }
        } while (choice != 0);

        input.close();
    }
}
```

# Module 3 / CO3 — Methods, Recursion & Arrays

Break the solution into reusable processing steps

### Methods

readReading(), calculateNet(), calculateValue(), printReport(), monthlySummary(). Each method has one responsibility.

### Arrays

For a simple version, store daily readings in double[] dailyGeneration and calculate total and average.

### Recursion

A recursive method can sum an array segment for demonstration. Loops are usually clearer for everyday reporting.

## Standard processing patterns

- Sum: add each reading to a running total.
- Average: total ÷ valid reading count; check that count is not zero.
- Maximum: compare each reading with the current maximum.

# CO3 — Methods, Recursion & Arrays (Part 1 of 3)

```
import java.util.Scanner;

public class Module3_MethodsRecursionArrays {
    static double calculateNet(double generated, double consumed) {
        return generated - consumed;
    }

    static double calculateTotal(double[] readings) {
        double total = 0;
        for (double reading : readings) {
            total += reading;
        }
        return total;
    }

    static double calculateAverage(double[] readings) {
        if (readings.length == 0) {
            return 0;
        }
        return calculateTotal(readings) / readings.length;
    }

    static void printReport(double[] readings) {
        System.out.println("\\nDaily generation readings:");
        for (int i = 0; i < readings.length; i++) {
            System.out.printf("Day %d: %.2f kWh%n", i + 1, readings[i]);
        }
        System.out.printf("Total: %.2f kWh%n", calculateTotal(readings));
        System.out.printf("Average: %.2f kWh%n", calculateAverage(readings));
```

# CO3 — Methods, Recursion & Arrays (Part 2 of 3)

```
                                                                    }

    // Recursive sum, included to demonstrate recursion.
    static double recursiveSum(double[] values, int index) {
        if (index == values.length) {
            return 0;
        }
        return values[index] + recursiveSum(values, index + 1);
    }

    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        System.out.print("How many daily readings? ");
        int count = input.nextInt();

        if (count <= 0) {
            System.out.println("Please enter a count greater than zero.");
            input.close();
            return;
        }

        double[] dailyGeneration = new double[count];
        for (int i = 0; i < count; i++) {
            System.out.print("Enter generation for day " + (i + 1) + ": ");
            dailyGeneration[i] = input.nextDouble();
            if (dailyGeneration[i] < 0) {
                System.out.println("Negative reading changed to 0.");
                dailyGeneration[i] = 0;
            }
```

# CO3 — Methods, Recursion & Arrays  (Part 3 of 3)

```
                                                                    }

        printReport(dailyGeneration);
        System.out.printf("Recursive total: %.2f kWh%n",
                recursiveSum(dailyGeneration, 0));
        System.out.printf("Example net energy (12.5 generated, 8.0 consumed): %.2f kWh%n",
                calculateNet(12.5, 8.0));

        input.close();
    }
}
```

# Module 4 / CO4 — Classes, Objects & Exceptions

Model the solar-monitor domain with robust Java classes

### EnergyReading

Fields: date, generatedKwh, consumedKwh, tariff
Methods: getNetKwh(), getEstimatedValue(), toString()

### SolarMonitor

Owns a list of readings
Methods: addReading(), findByDate(), getSummary(), saveData()

### Encapsulation

Make fields private; validate values in constructors/setters; expose safe methods instead of direct field access.

## Relationships and exception handling

- A SolarMonitor has many EnergyReading objects (has-a relationship).
- Use try-catch for NumberFormatException and IOException; display helpful messages.
- Keep validation near the data model so invalid readings cannot enter the system.

# Module 5 / CO5 — Strings, Algorithms & File I/O

Search, compare, sort, and persist readings

### String / StringBuilder

Build report lines with StringBuilder. Use equals() to compare text; do not use == for String content.

### Search & sort

Linear search by date. Sort readings by generated kWh or date using Comparator and List.sort().

### File I/O

Write records with BufferedWriter and read them with BufferedReader. Handle IOException.

## Example saved record format

```
2026-10-14,12.5,8.0,5.0
```

Before parsing a row, check the number of fields and handle malformed numeric values.

# Module 6 / CO6 — Collections, Lambdas & Streams

Process the dataset declaratively

### Collections

Use List<EnergyReading> for flexible records and Map<String, Integer> for counts by month.

### Lambdas

Use Comparator.comparingDouble(EnergyReading::getGeneratedKwh) for sorting or a Predicate for filtering.

### Stream pipeline

Filter records, map to generated kWh, then sum or average. Keep pipelines readable.

## Illustrative Java stream

```
double totalGenerated = readings.stream()\n    .mapToDouble(EnergyReading::getGeneratedKwh)\n
.sum();\n\nlong lowDays = readings.stream()\n    .filter(r -> r.getGeneratedKwh() < 5.0)\n    .count();
```

# How All Six Modules Work Together

Each course outcome contributes to one complete project

| | |
|---|---|
| **CO1 — Java foundations** | Read and calculate energy values using types, operators, Scanner, and formatted output. |
| **CO2 — Control flow** | Validate values, choose menu actions, and repeat until Exit. |
| **CO3 — Methods & arrays** | Organize calculations and generate daily or weekly summaries. |
| **CO4 — OOP & exceptions** | Model readings and monitor objects; protect data and handle errors. |
| **CO5 — Text & files** | Format reports, search/sort records, and save/load data. |
| **CO6 — Collections & streams** | Store many readings and filter, count, sort, and summarize efficiently. |

# Testing & Expected Results

Check normal cases, boundary cases, and invalid input

| Test case | Input / action | Expected result |
|---|---|---|
| **Normal reading** | Generated 12.5, consumed 8.0, tariff ₹5 | Net = 4.5 kWh; value = ₹62.50 |
| **Equal energy** | Generated 6.0, consumed 6.0 | Net = 0.0 kWh |
| **Low generation** | Generated 2.0 kWh | Low-generation label / warning |
| **Invalid input** | Negative value or text instead of number | Reject input and prompt again |
| **Empty summary** | No readings saved | Display “No records available” |
| **File error** | Missing or unreadable data file | Show clear error and continue safely |

Use consistent units (kWh) and display rounded values clearly.

# Conclusion & Future Enhancements

The Rooftop Solar Energy Monitor shows how Java concepts can solve a practical energy-tracking problem.

### What the project delivers

- Accurate calculations
- Validated inputs and safer error handling
- Searchable records and saved reports
- Daily/monthly energy summaries

### Future enhancements

- Live inverter / IoT sensor integration
- Charts and dashboard interface
- Alerts for unusual generation
- Database storage and user login

## THANK YOU

Questions?